

# Relatório de Mapeamento, Coleta e Homologação (Fases 1 a 3)

Este relatório consolida todas as etapas técnicas executadas até o momento no projeto de migração e reestruturação do NextERP (GCP → Hostinger VPS). Ele atua como histórico de engenharia reversa, análise de incidentes de homologação e guia de melhoria contínua.

---

## Sumário

1. Introdução e Objetivo do Projeto
  2. Fase 1: Investigação e Mapeamento na GCP
    - 2.1 Restabelecimento do Acesso SSH Seguro
    - 2.2 Descoberta da Arquitetura do Docker Compose
    - 2.3 Análise do Servidor Web Caddy
    - 2.4 Descoberta dos Fluxos do Extrator e Backups
  3. Fase 2: Geração de Backups e Coleta Segura
    - 3.1 Estratégia de Dumps e Compressão Nativos
    - 3.2 Padronização Semântica de Arquivos
    - 3.3 Transferência de Baixo Impacto
  4. Fase 3: Homologação no Laboratório Local (openSUSE)
    - 4.1 Ocorrência 01: Conflito de Portas de Rede (Nginx Proxy Manager)
    - 4.2 Ocorrência 02: Ambiente Virtual Python Corrompido (PEP 668)
    - 4.3 Ocorrência 03: Erro de Acesso Negado (Sincronização de Senhas MariaDB)
    - 4.4 Execução Bem-Sucedida dos Extratores
  5. O Porquê desta Abordagem (Efetividade do Laboratório)
  6. Cronograma das Etapas Restantes (Hostinger VPS)
  7. Plano de Melhoria Contínua para o Ambiente
- 

## 1. Introdução e Objetivo do Projeto

O NextERP é o sistema de gestão central da CDC, rodando sobre a plataforma containerizada Frappe/ERPNext v15. O objetivo deste projeto é realizar a **migração segura com downtime reduzido** do servidor legado hospedado em uma VM da Google Cloud Platform (GCP) para uma VPS na Hostinger, que oferece melhor custo-benefício e recursos dedicados.

O foco das atividades de hoje foi realizar a engenharia reversa do ambiente antigo (que não possuía documentação atualizada), coletar os backups estruturados e homologar o funcionamento da base e dos scripts de integração no laboratório local (máquina openSUSE do desenvolvedor).

---

## 2. Fase 1: Investigação e Mapeamento na GCP

### 2.1 Restabelecimento do Acesso SSH Seguro

O acesso inicial à VM prod1 no IP 35.184.131.8 falhou devido ao bloqueio do agente de chaves do Google (OS Login). \* **Ação:** Acessamos o Google Cloud Shell e adicionamos a chave pública id\_ed25519 local diretamente nas propriedades da VM, desativando temporariamente o OS Login

(enable-oslogin=FALSE). \* **Comportamento da GCP:** Para aplicar a alteração, realizamos um *Graceful Shutdown* (desligamento controlado) para preservar o banco de dados. Ao iniciar a VM, o GCP atribuiu um novo IP externo dinâmico: **136.113.22.112**. O acesso SSH foi restabelecido com sucesso como `dxcdc@136.113.22.112`.

## 2.2 Descoberta da Arquitetura do Docker Compose

Analisando os contêineres e logs na VM, localizamos a pasta `/home/gt_transformadigital/frappe_docker` e o arquivo **`pwd.yml`** que orquestra a aplicação: \* **Serviços:** Backend (Gunicorn), MariaDB 10.6, Redis (cache, queue e socketio), Nginx Frontend, Workers (short e long) e Scheduler. \* **Segredos:** A senha do banco de dados `root` estava exposta como `admin`. \* **Isolamento:** O banco de dados e o Redis estão estritamente isolados na rede privada `frappe_network`, sem portas expostas diretamente para o host.

## 2.3 Análise do Servidor Web Caddy

Ao mapear as portas do host com `ss -tulpn`, identificamos que as portas 80 e 443 não eram controladas por Nginx ou Apache, mas sim pelo **Caddy**. \* O arquivo `/etc/caddy/Caddyfile` revelou o subdomínio oficial ativo: **`estoque.cdc.org.br`**. \* Ele faz proxy reverso apontando para o contêiner frontend do Docker na porta 8080.

## 2.4 Descoberta dos Fluxos do Extrator e Backups

Varrendo a pasta do usuário e a tabela cron (`crontab -l`), encontramos duas automações ocultas gerenciadas pelo `root`: \* **Backup (`bkp.py`)**: Roda duas vezes ao dia. Ele executa o dump nativo do site frontend e envia para o Google Drive (`token.pickle`) utilizando autenticação OAuth. \* **Extrator (`run_job.sh`)**: Um pipeline sequencial de 5 scripts Python que roda de hora em hora. Ele consome a API do sistema externo **ONGSYS** via HTTPS (porta 443) em `www.ongsys.com.br` e grava os resultados no ERPNext local. \* **Conformidade Técnica:** Verificamos com `grep` que o pacote `pyodbc` é código morto nos scripts. **Não há dependência de drivers ODBC no Linux**, simplificando a containerização na Hostinger.

---

# 3. Fase 2: Geração de Backups e Coleta Segura

## 3.1 Estratégia de Dumps e Compressão Nativos

Utilizando o container de backend, geramos um backup atômico e consistente de produção:

```
sudo docker exec -it frappe_docker-backend-1 bench --site frontend backup --with-files
```

Iso gerou a base de dados SQL (10.9 MB), os arquivos públicos (530 KB), os arquivos privados (40 KB) e as configurações de chaves criptográficas (`site_config.json`).

## 3.2 Padronização Semântica de Arquivos

Para evitar confusões com os timestamps gerados pelo sistema, os arquivos foram renomeados localmente de forma semântica: \* `gcp-prod-database.sql.gz` \* `gcp-prod-public-files.tar` \* `gcp-prod-private-files.tar` \* `gcp-prod-site-config.json`

## 3.3 Transferência de Baixo Impacto

Os pacotes e as pastas de scripts foram copiados para a pasta `/tmp` da VM, tiveram suas permissões de dono ajustadas para o usuário `dxcdc` e foram baixados via `scp` seguro diretamente para a pasta `/backups`

da máquina local.

---

## 4. Fase 3: Homologação no Laboratório Local (openSUSE)

Durante a subida do laboratório na máquina openSUSE local, enfrentamos e mitigamos três problemas críticos de ambiente que entraram para o nosso histórico de lições aprendidas:

### 4.1 Ocorrência 01: Conflito de Portas de Rede (Nginx Proxy Manager)

- **Sintoma:** Falha ao iniciar o container frontend devido a porta 8080 já estar alocada.
- **Causa:** O host local já possuía uma instância ativa do *Nginx Proxy Manager* rodando em outra tarefa e escutando na 8080.
- **Solução:** Mapeamos a porta externa do ERPNext no `pwd.yml` local para **8085** e ajustamos o `configs.json` do extrator. O ERPNext abriu com sucesso em `http://localhost:8085`.

### 4.2 Ocorrência 02: Ambiente Virtual Python Corrompido (PEP 668)

- **Sintoma:** O pip falhou ao rodar o `run_job.sh` do extrator acusando ambiente gerenciado externamente.
- **Causa:** A pasta `venv` importada do GCP pertencia à arquitetura e binários do Debian, quebrando a ativação no openSUSE.
- **Solução:** Deletamos a pasta `venv` e permitimos que o interpretador local do openSUSE criasse um ambiente virtual limpo.

### 4.3 Ocorrência 03: Erro de Acesso Negado (Sincronização de Senhas MariaDB)

- **Sintoma:** API retornando Erro 500 devido a `pymysql.err.OperationalError: (1045, "Access denied")`.
- **Causa:** A subida inicial criou uma senha aleatória para o banco de dados. Ao injetarmos o `site_config.json` de produção com a senha antiga (`4aeed6qdlEJYeylN`), houve dessincronização com o usuário `root` do MariaDB.
- **Solução:** Acessamos o banco de dados local e redefinimos a senha do usuário do site para coincidir com a chave de produção.

### 4.4 Execução Bem-Sucedida dos Extratores

Após as correções, executamos o pipeline local do extrator. Todos os 5 scripts de integração rodaram de forma sequencial com sucesso absoluto, concluindo com **Código de Saída 0**.

---

## 5. O Porquê desta Abordagem (Efetividade do Laboratório)

Realizar as Fases 1 a 3 no laboratório local garantiu os seguintes benefícios: 1. **Garantia de Restore:** Provamos na prática que o backup de produção não está corrompido. 2. **Mitigação de Downtime:** Todos os erros de compatibilidade de bibliotecas Python, portas conflitantes e senhas de banco foram mitigados localmente sem afetar a produção GCP. 3. **Segurança de Processos:** A receita do Docker Compose e as chaves de API foram testadas com sucesso antes de serem expostas na VPS de destino.

---

## 6. Cronograma das Etapas Restantes (Hostinger VPS)

| <b>Etapa</b>   | <b>Ação Técnica</b>                                                             | <b>Tempo Est.</b> | <b>Impacto</b>  |
|----------------|---------------------------------------------------------------------------------|-------------------|-----------------|
| <b>Passo 1</b> | Acessar e configurar o Docker/Docker Compose na Hostinger VPS.                  | 30 min            | Nenhum          |
| <b>Passo 2</b> | Criar a estrutura do repositório, Nginx local e Caddy.                          | 30 min            | Nenhum          |
| <b>Passo 3</b> | Fazer o upload dos arquivos de backup (gcp-prod-*) e scripts.                   | 15 min            | Nenhum          |
| <b>Passo 4</b> | Executar a restauração definitiva na Hostinger.                                 | 15 min            | Nenhum          |
| <b>Passo 5</b> | Ativar o modo manutenção no GCP e tirar o backup incremental final (banco).     | 15 min            | <b>Downtime</b> |
| <b>Passo 6</b> | Importar o dump final na Hostinger e atualizar o apontamento de DNS do domínio. | 30 min            | <b>Downtime</b> |
| <b>Passo 7</b> | Configurar o Caddy para emissão automática do SSL e testar.                     | 15 min            | Fim da Janela   |

## 7. Plano de Melhoria Contínua para o Ambiente

Após a migração ser concluída na Hostinger, propomos as seguintes melhorias para aumentar a robustez e a segurança do NextERP:

- Remoção de Senhas em Texto Puro (Hardcoded):**
  - Mapear a senha do banco MariaDB (atualmente fixa como admin no pwd.yml) para ser lida a partir de uma variável secreta no arquivo .env.
- Containerização Completa do Extrator de Dados:**
  - Empacotar o script do Extrator de Dados e suas dependências Python em uma **imagem Docker dedicada** (com limite de recursos de CPU a 0.5 cores e 512 MB de RAM) integrada à rede Docker do Compose, eliminando a dependência do Cron do sistema operacional do host.
- Otimização de Performance do Extrator:**
  - O script 4\_Extrator\_produtos\_v2.py leva **4 minutos** para executar devido a iterações sequenciais. Implementar atualizações em lote (bulk update) ou multi-threading para reduzir o tempo de processamento para menos de 30 segundos.
- Notificações de Alerta de Backups no Mattermost:**
  - Integrar o script de backup (bkp.py) com um webhook silencioso do Mattermost para alertar a equipe imediatamente em caso de falha de upload no Google Drive.
- Criptografia Assimétrica de Backups (GPG):**
  - Implementar criptografia simétrica ou assimétrica (GPG) nos backups locais antes do upload para o Google Drive, garantindo proteção contra invasões na conta do Drive.